

WORKING DOCUMENT

LAMP Assistant

A custom AI for the LAMP diagnostic project: what it does, what it refuses to do, and what must happen before anyone relies on it.

Contents

1. Guide

2. Project knowledge template

Figures for growth, timelines and prices in this document are planning estimates, not measured research. Test them against your own results and adjust.

Guide

A custom AI assistant for a LAMP (loop-mediated isothermal amplification) diagnostic project. Four modes, one web app, built on the Claude API.

MODE	FOR	WHAT IT DOES
Ask the project	Lab and field staff	Answers protocol and troubleshooting questions from your own documented protocol
Read a result	Technicians	Reads a photo of tubes, or curve data, and returns a structured call per sample — checking controls first
Assay design review	R&D	Reviews LAMP primer sets (F3/B3/FIP/BIP/LF/LB) and target-region choice for detection assays
Explain to the public	Patients, farmers, community	Plain-language explanation of the disease and the test, in the user's language

What it will not do

These are enforced in the system prompt (`prompts.py`) and are deliberately not configurable from the UI:

- **No diagnosis.** It reports what was detected in a sample. It never tells a person what they have, what will happen to them, or what treatment to take.
- **Not a medical device.** Every surface says so. It is decision support and an educational tool, and it routes clinical decisions to a clinician and confirmation to an accredited lab.
- **Controls decide the run.** If the negative control reads positive or the positive control failed, the run is invalid and no sample call may be reported from it, however clean the sample tubes look. This is the single most common way a field result goes wrong, so it is a hard rule in the prompt *and* a required `run_valid` field in the response schema.
- **Detection only.** It helps find a pathogen or marker. It declines requests to modify an organism, or to help anything evade a diagnostic test.
- **No invented facts.** It will not make up protocol values, primer sequences, temperatures, or citations. With no project file loaded, it says so and gives only general LAMP guidance.

Setup

```
pip install -r requirements.txt
export ANTHROPIC_API_KEY=sk-ant-...           # get one at console.anthropic.com
python app.py                                # http://localhost:5000
```

Then fill in `knowledge/project.md`. Until you do, the assistant has no protocol and will say so on every answer. That file is injected into the system prompt on every request and is the assistant's authoritative source — your target, your temperatures, your read-out rules, your control rules, and your measured performance figures. Nothing else needs editing to make this yours.

Layout

app.py	Flask app: /api/chat (streaming), /api/read-result (structured JSON)
prompts.py	Safety preamble + the four mode prompts
knowledge/project.md	YOUR protocol – fill this in
templates/index.html	Single-page UI, light/dark, works on a phone
tests/test_app.py	27 tests, no API calls

Tests

```
ANTHROPIC_API_KEY=test-key python -m pytest tests/ -q
```

The Claude client is stubbed, so the suite costs nothing and runs offline. It covers the safety preamble being present in every mode, history sanitising (including a planted `role: "system"` injection attempt), image validation, streaming, refusals, truncation, API errors, and the structured-reading contract.

Design notes

- **Model:** `claude-opus-5-5`, overridable with `LAMP_AI_MODEL`. Effort is set per mode in `app.py` — `xhigh` for design review, `high` for reading results and Q&A, `medium` for public explanations.
- **Caching:** the safety preamble and project knowledge are identical on every request and marked with `cache_control`, so they form a stable cache prefix. Small knowledge files fall under the minimum cacheable length and silently won't cache; that costs nothing but gains nothing either. A substantial protocol file will cache.
- **Structured output:** result reading uses a JSON schema rather than prose, so the UI colour-codes calls instead of pattern-matching text, and `run_valid` can't be quietly omitted.
- **Conversation history** is client-side and sanitised server-side; only `user / assistant` text turns are accepted, trimmed to the last 20 exchanges.

Before anyone relies on it

1. Fill in `knowledge/project.md` and have a second person check it against the written protocol.
2. **Run a validation set through it.** Take 30–50 archived runs you already know the answer to — including invalid ones and borderline ones — and compare the assistant's calls to the lab's. Count the disagreements and look at every one. An assistant that is wrong on hard cases is worse than none.
3. Decide and write down who may act on its output and who may not.
4. Serve it behind authentication and over HTTPS. `app.py` binds to localhost and is not hardened for public deployment; put it behind a real WSGI server (gunicorn, uvicorn) and a reverse proxy.
5. Check your regulatory position. In most jurisdictions, software that informs a diagnostic decision is regulated. This tool is built as decision support and says so, but that is a claim you have to stand behind locally.

Project knowledge template

Everything in this file is injected into the assistant's system prompt on every request, and the assistant treats it as the authoritative source for the project. **Replace every TODO below with your own validated values.** Delete any section that does not apply rather than leaving a guess in it.

While the TODOs are still here, the assistant will say it has no project protocol and fall back to general LAMP guidance. That is the intended behaviour: it must not invent your protocol.

1. What this project is

- **Target disease / organism:** TODO
- **Target gene or region:** TODO
- **Sample type:** TODO (e.g. whole blood, nasal swab, plant leaf tissue, water)
- **Setting:** TODO (e.g. district clinic, field use by extension officers)
- **Stage:** TODO (bench development / field validation / routine use)
- **Intended use statement:** TODO. State plainly whether results are for research, screening with lab confirmation, or something else.

2. Assay

- **Primer set name / version:** TODO
- **Primer set source:** TODO (published paper with citation, or designed in-house — say which, and whether it has been validated)
- **Reaction volume:** TODO
- **Incubation temperature and time:** TODO (e.g. 65 °C for 30 min)
- **Enzyme / master mix:** TODO
- **Read-out:** TODO (colorimetric pH dye, hydroxynaphthol blue, fluorescence, turbidity, lateral flow)
- **Sample preparation:** TODO (extraction kit, or direct/boil-prep method)

3. How to read a result

This is the section the result-reading mode depends on most. Be precise.

- **Positive looks like:** TODO (e.g. "pink to yellow colour change")
- **Negative looks like:** TODO
- **Time point to read at:** TODO
- **Fluorescence cut-off / time-to-positive threshold:** TODO
- **Borderline / inconclusive appearance:** TODO
- **Known false-positive causes:** TODO (e.g. carry-over contamination, reading past the end point, non-specific amplification)

Controls, every run

- **Negative / no-template control:** TODO — expected appearance

- **Positive control:** TODO — expected appearance
- **Rule:** if the negative control reads positive or the positive control fails, the whole run is invalid and no sample result may be reported from it.
- **Internal / extraction control, if any:** TODO

4. Performance, as measured by this project

Only put numbers here that your own project measured or that come from a cited study on this exact assay. Leave it blank rather than filling it with estimates.

- **Limit of detection:** TODO
- **Sensitivity vs. reference method:** TODO (and the reference method)
- **Specificity vs. reference method:** TODO
- **Cross-reactivity tested against:** TODO
- **Sample size and setting of that evaluation:** TODO

5. Workflow and rules

- **Who runs the test:** TODO
- **What happens on a positive:** TODO (referral, confirmatory test, reporting)
- **What happens on an invalid run:** TODO
- **Notifiable-disease reporting obligations:** TODO
- **Contamination control:** TODO (separate pre/post-amplification areas, etc.)

6. Glossary for the public-explainer mode

How the project wants the disease and the test described to non-specialists, in each language it works in.

- **Disease, in plain words:** TODO
- **The test, in plain words:** TODO
- **What a positive means for the person:** TODO
- **What a negative means for the person:** TODO
- **Languages to answer in:** TODO